

LAPORAN PROJECT

Implementasi Teknik Pengolahan Citra untuk Peningkatan Kualitas dan Analisis Gambar Digital Produk UMKM



Nama: Putri Melati Ramadhaniati

NIM: 312410194

Kelas: I241E

Mata Kuliah: Pengolahan Citra Digital

Dosen Pengampu: Dr. Muhamad Fatchan, S.Kom., M.Kom

UNIVERSITAS PELITA BANGSA

FAKULTAS TEKNIK

2026

BAB I

PENDAHULUAN

1.1 Latar Belakang

Sekarang ini hampir semua orang bisa foto produk sendiri pakai hp, upload ke marketplace, dan berharap dagangannya laku. Tapi nggak semua foto produk itu punya kualitas yang bagus, apalagi buat pelaku UMKM yang mungkin nggak punya kamera mahal atau studio foto. Nah, di sinilah pengolahan citra digital bisa jadi solusi yang menarik untuk dicoba.

Pengolahan citra digital itu intinya adalah cara kita 'mengotak-atik' gambar menggunakan program komputer. Bisa buat memperjelas gambar yang buram, menganalisis warna, mendeteksi bentuk objek, sampai memisahkan objek dari latar belakangnya. Semua itu bisa dilakukan secara otomatis dengan kode program.

Di project mini ini, saya mencoba membuat aplikasi sederhana berbasis Python yang bisa melakukan beberapa proses pengolahan citra pada gambar produk UMKM. Dataset yang dipakai adalah 20 foto produk yang sudah saya siapkan. Aplikasinya dibuat pakai Tkinter supaya ada tampilan GUI-nya, jadi lebih mudah digunakan dan nggak harus lewat terminal terus.

1.2 Rumusan Masalah

Dari latar belakang di atas, ada beberapa hal yang ingin saya jawab lewat project ini:

- Gimana cara mengimplementasikan teknik pengolahan citra digital pada gambar produk UMKM menggunakan Python?
- Teknik apa saja yang cocok dipakai untuk meningkatkan kualitas dan menganalisis gambar produk?
- Bagaimana merancang aplikasi GUI yang gampang dipakai meski tanpa latar belakang coding?

1.3 Tujuan

Secara garis besar, tujuan dari project ini adalah:

- Memahami konsep dasar pengolahan citra digital lewat praktik langsung.
- Membuat aplikasi pengolahan citra berbasis GUI menggunakan Python dan Tkinter.
- Menerapkan minimal 5 teknik pengolahan citra yang berbeda pada dataset produk UMKM.
- Melihat dan menganalisis perbedaan hasil dari masing-masing teknik yang dipakai.

1.4 Manfaat

Harapannya project ini bisa bermanfaat untuk:

- Menambah pemahaman saya sendiri tentang pengolahan citra secara praktis, bukan cuma teori.
- Menghasilkan aplikasi yang bisa dipakai langsung untuk memproses foto produk UMKM.
- Jadi referensi awal kalau mau dikembangkan lebih lanjut, misalnya ditambah fitur ML.

BAB II

DASAR TEORI

2.1 Pengolahan Citra Digital

Pengolahan citra digital (Digital Image Processing) adalah proses memanipulasi gambar menggunakan algoritma komputer. Tujuannya bisa bermacam-macam: meningkatkan kualitas visual, mengekstrak informasi tertentu dari gambar, atau mempersiapkan gambar untuk proses analisis berikutnya. Secara teknis, citra digital direpresentasikan sebagai matriks dua dimensi yang tiap elemennya disebut piksel.

Yang menarik dari pengolahan citra adalah hasilnya bisa langsung kelihatan, jadi kita bisa langsung tahu apakah teknik yang dipakai sudah sesuai atau belum.

2.2 Konversi Citra

Konversi citra artinya mengubah representasi warna gambar dari satu format ke format lain. Dua jenis konversi yang saya pakai di project ini adalah:

Pertama, RGB ke Grayscale. Caranya dengan menggabungkan tiga channel warna (merah, hijau, biru) menjadi satu nilai intensitas keabuan. Rumus yang umum dipakai adalah: $Y = 0.299R + 0.587G + 0.114B$. Bobotnya beda-beda karena mata manusia lebih sensitif terhadap warna hijau.

Kedua, RGB ke Biner. Gambar diubah jadi hanya dua nilai: hitam (0) atau putih (255). Di sini saya pakai metode Otsu Thresholding yang bisa otomatis menentukan nilai ambang batas terbaik berdasarkan histogram gambar, jadi nggak perlu tebak-tebak manual.

2.3 Perbaikan Kualitas Citra

Ada dua teknik perbaikan kualitas yang saya implementasikan:

Histogram Equalization bekerja dengan cara meratakan distribusi nilai intensitas piksel di seluruh gambar. Hasilnya, gambar yang tadinya terlalu gelap atau terlalu terang jadi lebih seimbang dan detailnya lebih kelihatan.

Contrast Stretching sedikit berbeda. Teknik ini 'meregangkan' rentang nilai piksel supaya mengisi seluruh range 0-255. Saya pakai nilai persentil ke-2 dan ke-98 supaya piksel-piksel ekstrem (outlier) nggak terlalu mempengaruhi hasilnya. Efeknya lebih natural dibanding Histogram Equalization.

2.4 Filtering

Filtering adalah operasi yang mengubah nilai piksel berdasarkan nilai piksel-piksel di sekitarnya (tetangga), menggunakan proses yang disebut konvolusi. Ada tiga filter yang saya pakai:

Mean Filter: mengganti setiap piksel dengan rata-rata piksel tetangganya. Sederhana dan cepat, tapi hasilnya agak blur karena nggak membedakan piksel tepi dengan piksel biasa.

Median Filter: mengganti piksel dengan nilai tengah (median) dari tetangganya. Lebih bagus dari Mean Filter kalau ada noise acak, karena nilai ekstrem nggak ikut dihitung rata-rata.

Gaussian Filter: menggunakan bobot berdasarkan distribusi Gaussian, jadi piksel yang lebih dekat ke pusat kernel dapat bobot lebih besar. Hasilnya blur-nya lebih halus dan terlihat lebih natural.

2.5 Deteksi Tepi

Deteksi tepi (edge detection) dipakai untuk menemukan batas-batas objek dalam gambar, yaitu area di mana terjadi perubahan intensitas yang cukup signifikan. Dua metode yang saya pakai:

Sobel: menggunakan dua kernel 3x3 untuk menghitung gradien horizontal dan vertikal. Hasilnya digabungkan dengan rumus magnitude: $G = \sqrt{G_x^2 + G_y^2}$. Cukup tahan terhadap noise dan mudah diimplementasikan.

Canny: ini yang lebih canggih. Ada beberapa tahap di dalamnya — penghalusan Gaussian, perhitungan gradien, non-maximum suppression, lalu hysteresis thresholding. Hasilnya tepi yang terdeteksi lebih tipis dan lebih akurat dibanding Sobel.

2.6 Segmentasi Citra

Segmentasi adalah proses membagi gambar menjadi beberapa bagian (region) berdasarkan kesamaan tertentu, misalnya warna atau intensitas. Dua metode yang saya pakai:

Threshold Segmentation: cara paling sederhana. Piksel yang nilainya di atas threshold masuk satu grup, sisanya masuk grup lain. Di sini saya pakai `threshold = 127` dengan flag `THRESH_BINARY_INV`.

K-Means Segmentation: lebih kompleks. Piksel-piksel dikelompokkan ke dalam K cluster berdasarkan kemiripan warnanya. Saya pakai `K=3`, artinya gambar dibagi jadi 3 region warna dominan. Teknik ini lebih cocok untuk analisis distribusi warna pada gambar produk.

2.7 Library yang Digunakan

- OpenCV (cv2) library utama untuk semua operasi pengolahan citra. Hampir semua fungsi inti ada di sini.
- NumPy untuk operasi matematika pada array gambar, terutama di bagian Contrast Stretching dan Sobel.
- Pillow (PIL) dipakai untuk konversi format gambar supaya bisa ditampilkan di Tkinter.
- Tkinter library bawaan Python untuk membuat GUI. Dipilih karena ringan dan nggak perlu install tambahan.

BAB III

METODOLOGI

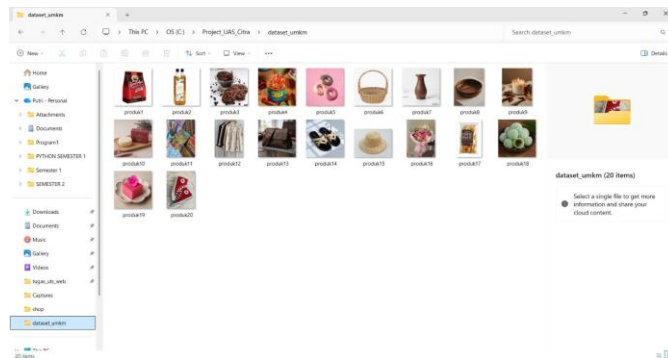
3.1 Alur Pengerjaan Project

Pengerjaan project ini saya lakukan secara bertahap. Nggak langsung nulis kode dari awal, tapi direncanakan dulu supaya nggak bolak-balik revisi terlalu banyak.

1. Menyiapkan dataset: mengumpulkan 20 foto produk UMKM dalam format JPG dan menyimpannya di folder dataset_umkm.
2. Merancang struktur aplikasi: menentukan fitur apa saja yang perlu ada, bagaimana tampilan GUI-nya, dan bagaimana alur kerjanya.
3. Menulis kode program: mengimplementasikan semua teknik pengolahan citra dan GUI secara bertahap.
4. Pengujian: menjalankan tiap fungsi dengan beberapa gambar berbeda untuk melihat hasilnya.
5. Analisis hasil: membandingkan dan mengevaluasi output dari masing-masing teknik.

3.2 Dataset

Dataset yang dipakai terdiri dari 20 gambar produk UMKM yang saya simpan di folder dataset_umkm dengan nama file produk1.jpg sampai produk20.jpg. Gambar-gambar ini mencakup berbagai jenis produk konsumen seperti makanan, minuman, dan produk sehari-hari. Format file semuanya JPG.



Gambar: 20 Gambar Dataset Produk UMKM

Pemilihan tema produk UMKM ini sengaja dilakukan karena relevan dengan kondisi nyata banyak pelaku UMKM yang butuh bantuan dalam menampilkan produknya secara visual lebih baik.

3.3 Spesifikasi yang Digunakan

- Sistem Operasi: Windows 11
- Bahasa Pemrograman: Python 3.x
- Editor: Visual Studio Code
- Library: OpenCV, NumPy, Pillow, Tkinter
- GUI Framework: Tkinter (sudah bawaan Python, tidak perlu install terpisah)

3.4 Struktur Aplikasi

Aplikasi dibuat dengan pendekatan OOP (Object-Oriented Programming). Semua komponen GUI dan logika pengolahan citra dikumpulkan dalam satu kelas bernama AplikasiPengolahanCitra. Kelas ini diinisialisasi dengan window Tkinter sebagai parameter utamanya.

Tampilannya terbagi dua bagian: panel kiri untuk tombol-tombol menu, dan panel kanan untuk menampilkan gambar asli vs hasil proses secara berdampingan. Di bagian bawah ada status bar yang kasih info tentang proses yang lagi aktif.

BAB IV

IMPLEMENTASI

4.1 Struktur Direktori Project

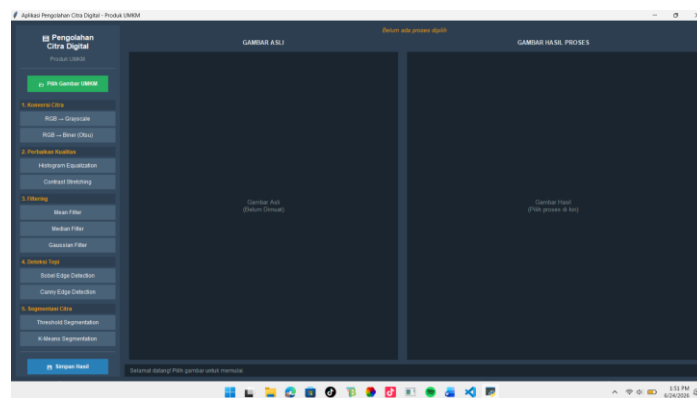
Struktur folder project yang digunakan adalah sebagai berikut:

```
Project_UAS_Citra/  
  dataset_umkm/  
    produk1.jpg  
    produk2.jpg  
    ... (hingga produk20.jpg)  
  uas_citra_umkm.py
```

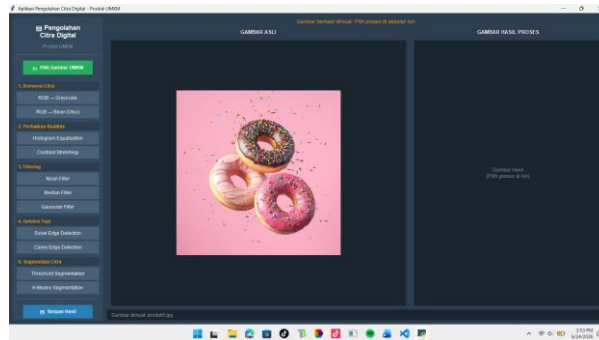
4.2 Implementasi GUI

Bagian GUI dibuat pakai Tkinter dengan tema warna gelap supaya tampilannya lebih modern. Panel kiri berisi semua tombol yang dikelompokkan berdasarkan kategori proses: Konversi, Perbaikan Kualitas, Filtering, Deteksi Tepi, dan Segmentasi. Panel kanan punya dua label gambar yang ditampilkan berdampingan kiri untuk gambar asli, kanan untuk hasil proses.

Ada beberapa fitur tambahan yang saya tambahkan supaya aplikasinya lebih lengkap: label nama proses yang aktif (warna oranye di atas), status bar bawah yang kasih info tiap selesai proses, dan tombol Simpan Hasil untuk menyimpan output ke file.



Gambar: Tampilan awal aplikasi setelah dijalankan (sebelum gambar dipilih)



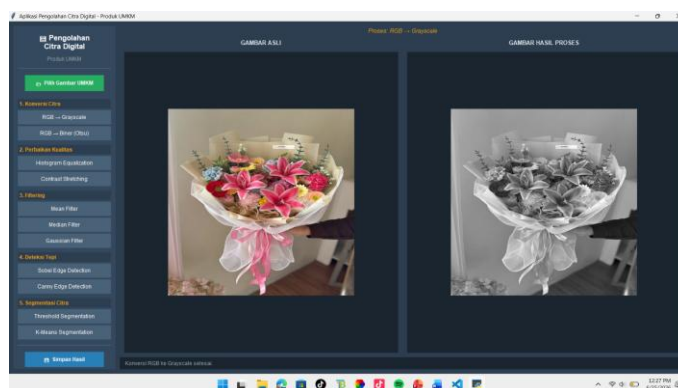
Gambar: Tampilan aplikasi setelah memilih gambar produk dari folder dataset_umkm

4.3 Implementasi Tiap Teknik Pengolahan Citra

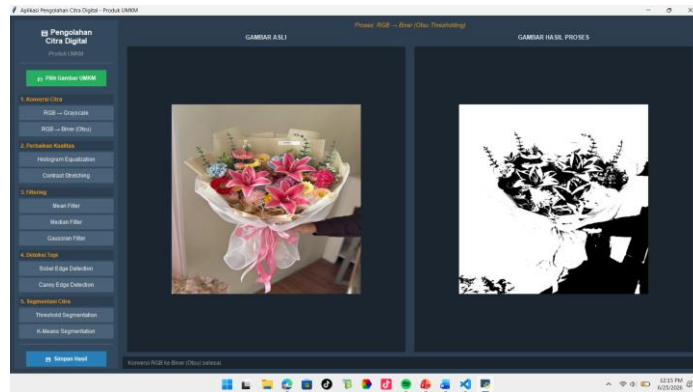
4.3.1 Konversi Citra

Untuk konversi RGB ke Grayscale, saya pakai fungsi `cv2.cvtColor()` dengan parameter `cv2.COLOR_BGR2GRAY`. Perlu diingat bahwa OpenCV membaca gambar dalam format BGR, bukan RGB, jadi parameternya harus sesuai.

Untuk konversi ke Biner, saya pakai `cv2.threshold()` dengan kombinasi flag `cv2.THRESH_BINARY + cv2.THRESH_OTSU`. Dengan Otsu, nilai threshold-nya dihitung otomatis dari histogram gambar, jadi hasilnya lebih konsisten untuk berbagai jenis gambar.



Gambar: Hasil konversi RGB ke Grayscale pada salah satu gambar produk

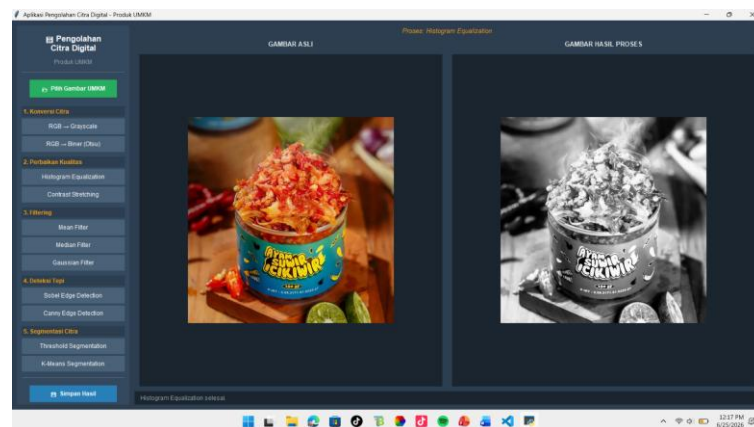


Gambar: Hasil konversi RGB ke Biner (Otsu Thresholding) pada gambar yang sama

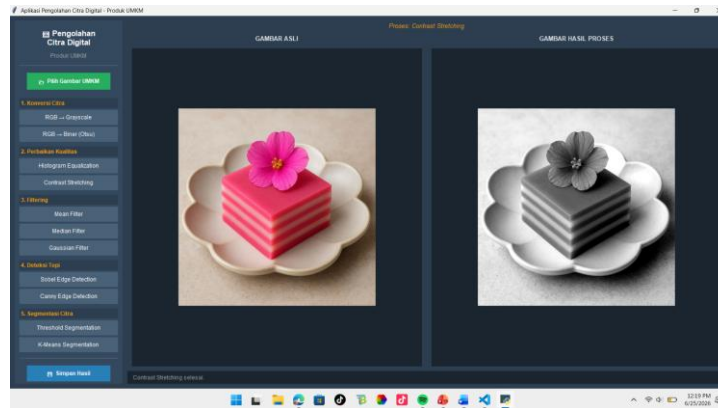
4.3.2 Perbaikan Kualitas Citra

Histogram Equalization diimplementasikan dengan `cv2.equalizeHist()` yang diaplikasikan pada citra grayscale. Fungsi ini langsung meratakan distribusi histogram secara otomatis.

Contrast Stretching saya tulis secara manual karena OpenCV tidak punya fungsi langsung untuk ini. Caranya: ambil nilai persentil ke-2 dan ke-98 dari gambar grayscale, lalu petakan ulang semua nilai piksel ke rentang 0-255 secara linear. Pakai `np.clip()` supaya nilainya nggak keluar dari range.



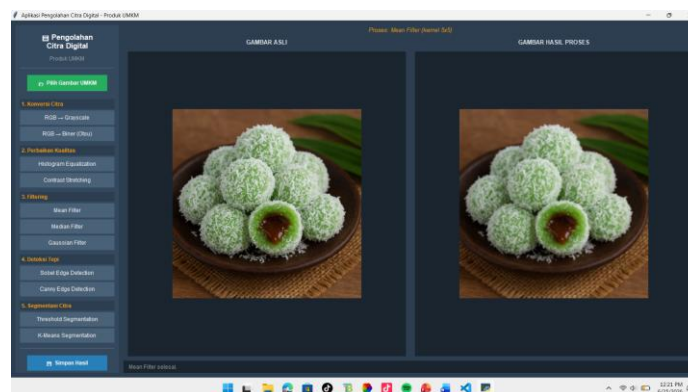
Gambar: Perbandingan gambar asli vs hasil Histogram Equalization



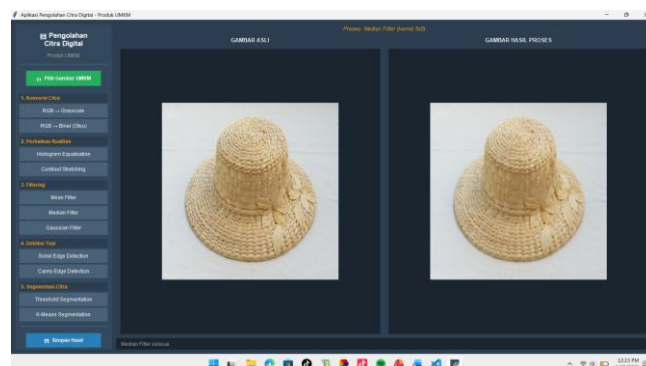
Gambar: Perbandingan gambar asli vs hasil Contrast Stretching

4.3.3 Filtering

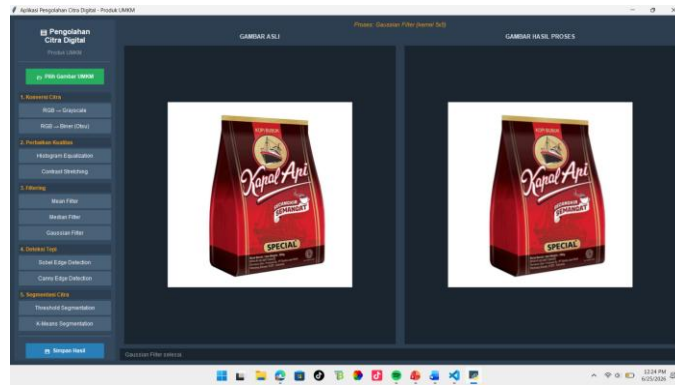
Mean Filter pakai `cv2.blur()` dengan kernel 5x5. Median Filter pakai `cv2.medianBlur()` juga dengan ukuran kernel 5. Gaussian Filter pakai `cv2.GaussianBlur()` dengan kernel 5x5 dan $\sigma = 0$ (artinya sigma dihitung otomatis oleh OpenCV berdasarkan ukuran kernel).



Gambar: Hasil Mean Filter



Gambar: Hasil Median Filter

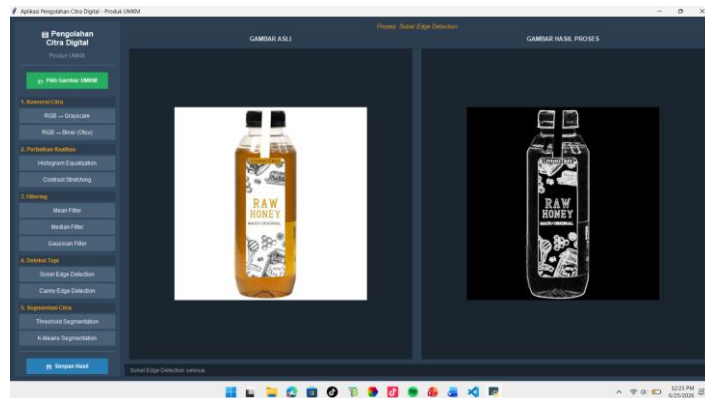


Gambar: Hasil Gaussian Filter

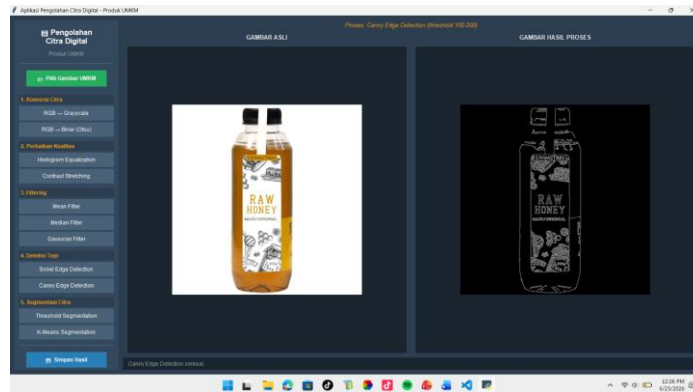
4.3.4 Deteksi Tepi

Untuk Sobel, saya hitung gradien arah X dan Y secara terpisah pakai `cv2.Sobel()` dengan tipe data `cv2.CV_64F` supaya bisa menangkap nilai negatif. Lalu keduanya digabungkan dengan rumus: $\text{magnitude} = \sqrt{G_x^2 + G_y^2}$, dan dikonversi balik ke `uint8` pakai `cv2.convertScaleAbs()`.

Canny diimplementasikan dengan `cv2.Canny()` dengan threshold bawah 100 dan threshold atas 200. Nilai ini dipilih setelah coba-coba beberapa kali dan hasilnya paling konsisten untuk gambar produk.



Gambar: Hasil Sobel Edge Detection pada salah satu gambar produk

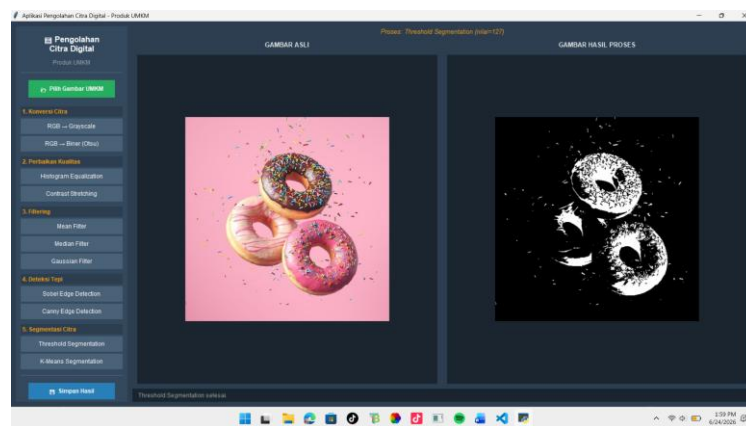


Gambar: Hasil Canny Edge Detection pada gambar yang sama untuk perbandingan

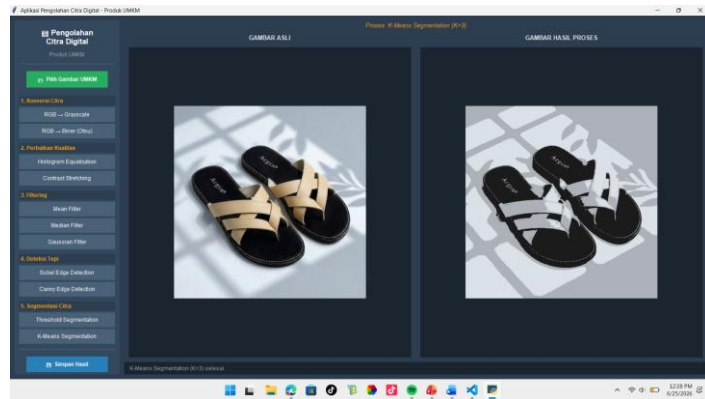
4.3.5 Segmentasi Citra

Threshold Segmentation pakai `cv2.threshold()` dengan nilai 127 dan flag `cv2.THRESH_BINARY_INV`, artinya piksel dengan nilai di atas 127 jadi hitam, sisanya jadi putih (dibalik dari biasanya).

K-Means Segmentation sedikit lebih kompleks. Gambar di-reshape dulu jadi array 2D dengan 3 kolom (RGB), lalu diproses dengan `cv2.kmeans()` dengan $K=3$. Setelah dapat label dan centroid, gambar di-reconstruct ulang dari nilai centroid masing-masing cluster.



Gambar: Hasil Threshold Segmentation (nilai threshold = 127)



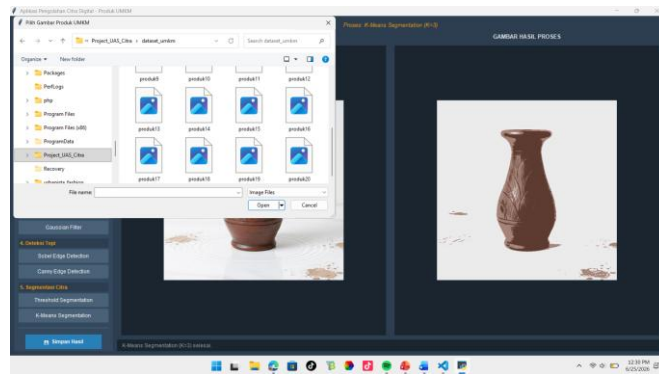
Gambar: Hasil K-Means Segmentation dengan K=3 cluster

BAB V

HASIL DAN PEMBAHASAN

5.1 Tampilan Aplikasi

Aplikasi berhasil jalan tanpa error. Tampilannya cukup rapi dengan dua panel utama yang jelas fungsinya masing-masing. Berikut tampilan aplikasi saat memilih gambar:



Gambar: Tampilan aplikasi saat sedang memilih gambar (total 20 gambar)

5.2 Hasil Tiap Teknik dan Pembahasannya

5.2.1 Konversi RGB ke Grayscale

Hasilnya cukup sesuai ekspektasi. Warna-warna pada gambar produk berhasil diubah jadi skala abu-abu, tapi informasi bentuk dan teksturnya masih terjaga. Yang menarik, produk dengan warna-warna kontras (misalnya merah pekat) punya distribusi keabuan yang lebih dramatis dibanding produk berwarna netral.

5.2.2 Konversi RGB ke Biner (Otsu)

Metode Otsu terbukti cukup pintar dalam menentukan threshold-nya sendiri. Untuk gambar dengan background yang kontras dengan produknya, hasilnya bagus objek dan background terpisah cukup jelas. Tapi untuk gambar yang warna produknya mirip dengan background, hasilnya kurang optimal karena thresholding memang punya keterbatasan di sini.

5.2.3 Histogram Equalization

Teknik ini paling terasa manfaatnya pada gambar yang agak gelap atau kurang kontras. Setelah equalization, detail-detail yang tadinya nggak kelihatan jadi lebih tampak. Tapi ada trade-off-nya: kadang gambar jadi terlihat sedikit 'berlebihan' atau noise-nya jadi lebih kelihatan karena kontrasnya ditingkatkan secara agresif.

5.2.4 Contrast Stretching

Dibanding Histogram Equalization, hasilnya lebih natural dan nggak terlalu dramatis. Ini karena penggunaan persentil yang menghindari piksel ekstrem jadi mempengaruhi keseluruhan gambar. Cocok buat gambar yang sudah cukup bagus tapi masih perlu sedikit perbaikan kontras.

5.2.5 Filtering (Mean, Median, Gaussian)

Ketiganya menghasilkan efek blur yang berbeda-beda karakter. Mean Filter paling 'kasar' semua area kena blur sama rata termasuk tepinya. Median Filter paling bagus dalam mempertahankan tepi objek sambil tetap menghaluskan area yang datar. Gaussian Filter paling halus hasilnya, blur-nya gradual dan terasa lebih alami.

Untuk keperluan preprocessing sebelum deteksi tepi atau segmentasi, Gaussian Filter yang paling sering saya rekomendasikan karena hasilnya paling stabil.

5.2.6 Sobel vs Canny Edge Detection

Keduanya berhasil mendeteksi tepi, tapi kualitasnya beda cukup jauh. Sobel menghasilkan tepi yang lebih 'tebal' dan kadang ada area yang terdeteksi padahal bukan tepi sebenarnya. Canny jauh lebih rapi tepinya tipis, bersih, dan cukup akurat. Wajar sih, karena Canny punya tahap filtering tambahan yang Sobel nggak punya.

5.2.7 Threshold vs K-Means Segmentation

Threshold Segmentation cocok kalau hanya butuh pemisahan sederhana antara dua region. Tapi kalau gambar produknya punya banyak variasi warna, hasilnya sering kurang memuaskan karena cuma bisa bikin dua kelompok.

K-Means dengan $K=3$ jauh lebih informatif. Tiga region warna yang dihasilkan biasanya bisa mewakili warna utama produk, warna aksen, dan background. Ini berguna kalau mau menganalisis distribusi warna pada desain kemasan produk UMKM.

5.3 Tabel Perbandingan Semua Teknik

Tabel 5.1 Ringkasan Teknik Pengolahan Citra yang Diimplementasikan

No	Teknik	Fungsi Utama	Catatan
1	RGB ke Grayscale	Konversi warna	Warna hilang, bentuk tetap
2	RGB ke Biner (Otsu)	Konversi warna	Threshold otomatis
3	Histogram Equalization	Perbaikan kontras	Agresif, cocok untuk foto gelap
4	Contrast Stretching	Perbaikan kontras	Lebih natural, pakai persentil
5	Mean Filter	Smoothing	Blur merata, cepat
6	Median Filter	Smoothing	Terbaik untuk salt-pepper noise
7	Gaussian Filter	Smoothing	Blur paling halus dan natural
8	Sobel Edge Detection	Deteksi tepi	Tepi tebal, ada noise
9	Canny Edge Detection	Deteksi tepi	Tepi tipis, akurat
10	Threshold Segmentation	Segmentasi	Sederhana, 2 region saja
11	K-Means Segmentation	Segmentasi	$K=3$, analisis warna dominan

BAB VI

KESIMPULAN DAN SARAN

6.1 Kesimpulan

Setelah mengerjakan project ini dari awal sampai selesai, ada beberapa hal yang bisa saya simpulkan:

6. Aplikasi pengolahan citra digital berbasis Python berhasil dibuat dan bisa dijalankan tanpa error. Total ada 11 teknik pengolahan citra yang diimplementasikan, lebih dari syarat minimal 5 teknik.
7. GUI berbasis Tkinter ternyata cukup fungsional dan nggak terlalu susah diimplementasikan. Tampilannya juga bisa dibuat cukup rapi dengan pengaturan warna dan layout yang tepat.
8. Masing-masing teknik punya kelebihan dan kekurangannya sendiri. Nggak ada yang 'terbaik' untuk semua kasus semuanya tergantung kebutuhan dan karakteristik gambar yang diproses.
9. Untuk analisis warna produk UMKM, K-Means Segmentation terbukti paling informatif. Sedangkan untuk preprocessing sebelum analisis lebih lanjut, Gaussian Filter dan Canny Edge Detection adalah pilihan yang solid.
10. Project ini membuktikan bahwa pengolahan citra digital bisa diimplementasikan secara praktis dan nggak harus pakai tools mahal Python dengan library gratisan sudah cukup.

6.2 Saran

Kalau project ini mau dikembangkan lebih lanjut, beberapa hal yang bisa ditambahkan antara lain:

- Tambahkan fitur batch processing supaya bisa memproses semua 20 gambar dataset sekaligus, bukan satu-satu.
- Integrasikan model Machine Learning atau CNN sederhana untuk klasifikasi jenis produk secara otomatis.
- Tambahkan visualisasi histogram sebelum dan sesudah setiap proses supaya analisisnya lebih lengkap.

- Buat versi web-based pakai Flask supaya bisa diakses dari browser tanpa perlu install Python.

DAFTAR PUSTAKA

- Bradski, G., & Kaehler, A. (2008). Learning OpenCV: Computer Vision with the OpenCV Library. O'Reilly Media.
- Gonzalez, R. C., & Woods, R. E. (2018). Digital Image Processing (4th ed.). Pearson.
- Szeliski, R. (2022). Computer Vision: Algorithms and Applications (2nd ed.). Springer.
- OpenCV Documentation. (2024). OpenCV 4.x Documentation. Diakses dari <https://docs.opencv.org/>
- NumPy Documentation. (2024). NumPy User Guide. Diakses dari <https://numpy.org/doc/>
- Pillow Documentation. (2024). Pillow (PIL Fork) Documentation. Diakses dari <https://pillow.readthedocs.io/>
- Python Software Foundation. (2024). tkinter — Python interface to Tcl/Tk. Diakses dari <https://docs.python.org/3/library/tkinter.html>
- Canny, J. (1986). A Computational Approach to Edge Detection. IEEE Transactions on Pattern Analysis and Machine Intelligence, 8(6), 679–698.
- Otsu, N. (1979). A Threshold Selection Method from Gray-Level Histograms. IEEE Transactions on Systems, Man, and Cybernetics, 9(1), 62–66.
- MacQueen, J. (1967). Some Methods for Classification and Analysis of Multivariate Observations. Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability, 1, 281–297.